Manual Técnico

PROGRAMACIÓN

Realizado por: Manuel Camacho Font

05/03/2026

ÍNDICE

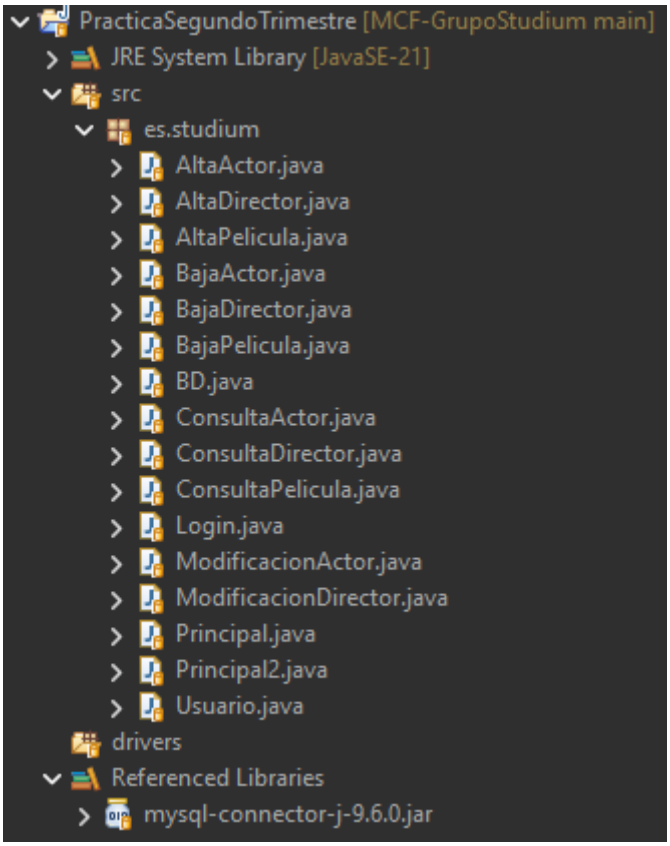
Presentación de la aplicación	Página 2
Herramientas Agile	Página 3
Diseño Base de Datos	Página 4
Construcción Base de Datos	Páginas 5 hasta 6
Usuario Base de Datos	Página 6
Diseño Funcional	Páginas 7 hasta 16
Repositorio GitHub	Página 17
Ejecución y Pruebas	Páginas 17 hasta 22
Referencias	Página 23

APLICACIÓN

Esta aplicación está diseñada en JAVA con la librería AWT, y se trata de un gestor de información conectado a una base de datos con el que podemos realizar las cuatro operaciones del CRUD (Create, Read, Update, Delete). Su nombre es Cinemateca por el hecho de que los datos con los que vamos a trabajar son del mundo del cine, trabajaremos con *Directores*, *Películas* y *Actores*.

La Base de Datos que hemos conectado (usando el conector de tipo 4 JDBC) con la aplicación ha sido creada en MySQL server, actualmente existen dos tipos de usuarios con acceso a dicha Base de Datos, uno básico que solo tendrá permisos de INSERT en nuestras tablas y visualmente solamente dispondrá de esas opciones, y un usuario administrador con los permisos de INSERT, SELECT, UPDATE y DELETE.

Esta aplicación ha sido desarrollada en eclipse, con JDK21. Para la gestión de sus clases he creado una clase general llamada BD por motivos de organización y escalabilidad en la que guardo datos de conexión y sentencias SQL, una clase llamada Usuario en la que se controla y gestiona las funciones que puede realizar cada Usuario tanto para acceder al sistema como en el manejo de la aplicación y luego ya tenemos las vistas de Login, Principales y clases específicas para cada desempeño del CRUD.

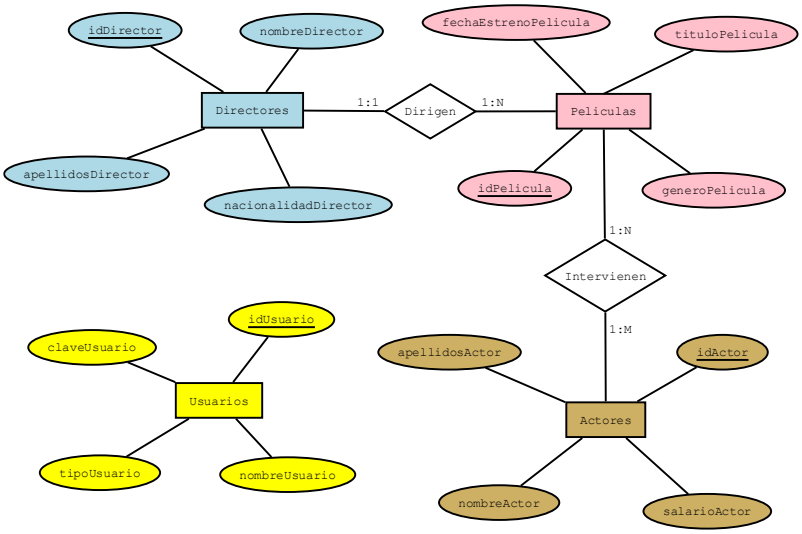


HERRAMIENTAS AGILE

Para la organización y la gestión de este proyecto he optado por el uso de una herramienta que ofrece GitHub, en la que puedes anotar diferentes tareas dentro de tu proyecto.

DISEÑO DE BASE DE DATOS

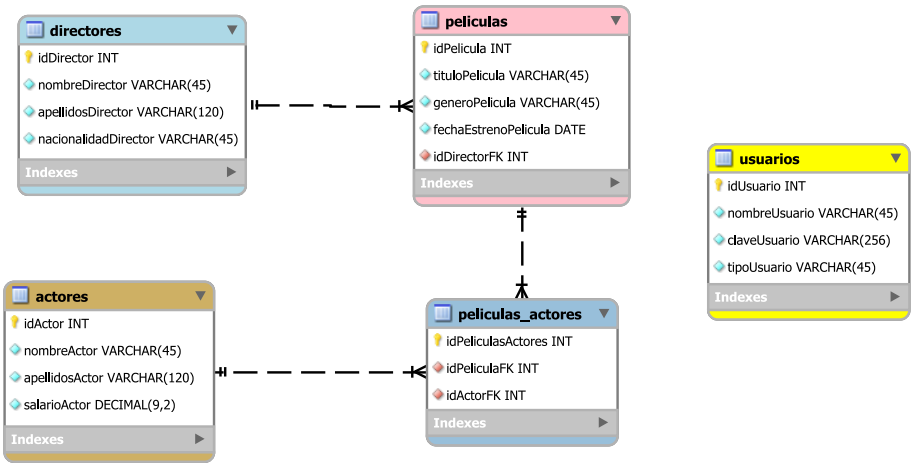
Diagrama Entidad – Relación



Esquema Relacional

```
directores(#idDirector, nombreDirector, apellidosDirector, nacionalidadDirector);
peliculas(#idPelicula, tituloPelicula, generoPelicula, fechaEstrenoPelicula, idDirectorFK);
actores(#idActor, nombreActor, apellidosActor,salarioActor);
peliculas_actores(#idPeliculasActores, idPeliculaFK, idActorFK);
usuarios(#idUsuario, nombreUsuario, claveUsuario, tipoUsuario);
```

Diagrama Workbench



CONSTRUCCIÓN DE BD

```sql
-- Crear BD
CREATE DATABASE p_cine
CHARSET utf8mb4
COLLATE utf8mb4_spanish2_ci;
-- Activar BD
USE p_cine;
-- Crear tabla directores
CREATE TABLE directores (
    idDirector INT AUTO_INCREMENT,
    nombreDirector VARCHAR(45) NOT NULL,
    apellidosDirector VARCHAR(120) NOT NULL,
    nacionalidadDirector VARCHAR(45),
    PRIMARY KEY (idDirector)
);
-- Crear tabla peliculas
CREATE TABLE peliculas (
    idPelicula INT AUTO_INCREMENT,
    tituloPelicula VARCHAR(45) NOT NULL,
    generoPelicula VARCHAR(45) NOT NULL,
    fechaEstrenoPelicula DATE NOT NULL,
    idDirectorFK INT NOT NULL,
    PRIMARY KEY (idPelicula),
    FOREIGN KEY (idDirectorFK)
        REFERENCES directores (idDirector)
);
-- Crear tabla actores
CREATE TABLE actores (
    idActor INT AUTO_INCREMENT,
    nombreActor VARCHAR(45) NOT NULL,
    apellidosActor VARCHAR(120) NOT NULL,
    salarioActor DECIMAL(9 , 2 ) NOT NULL,
    PRIMARY KEY (idActor)
);
```

```sql
-- Crear tabla peliculas_actores
CREATE TABLE peliculas_actores (
    idPeliculasActores INT AUTO_INCREMENT,
    idPeliculaFK INT NOT NULL,
    idActorFK INT NOT NULL,
    PRIMARY KEY (idPeliculasActores),
    FOREIGN KEY (idPeliculaFK)
        REFERENCES peliculas (idPelicula),
    FOREIGN KEY (idActorFK)
        REFERENCES actores (idActor)
);
-- Crear tabla usuarios
CREATE TABLE usuarios (
    idUsuario INT AUTO_INCREMENT,
    nombreUsuario VARCHAR(45) NOT NULL,
    claveUsuario VARCHAR(256) NOT NULL,
    tipoUsuario VARCHAR(45) NOT NULL,
    PRIMARY KEY (idUsuario)
);
```

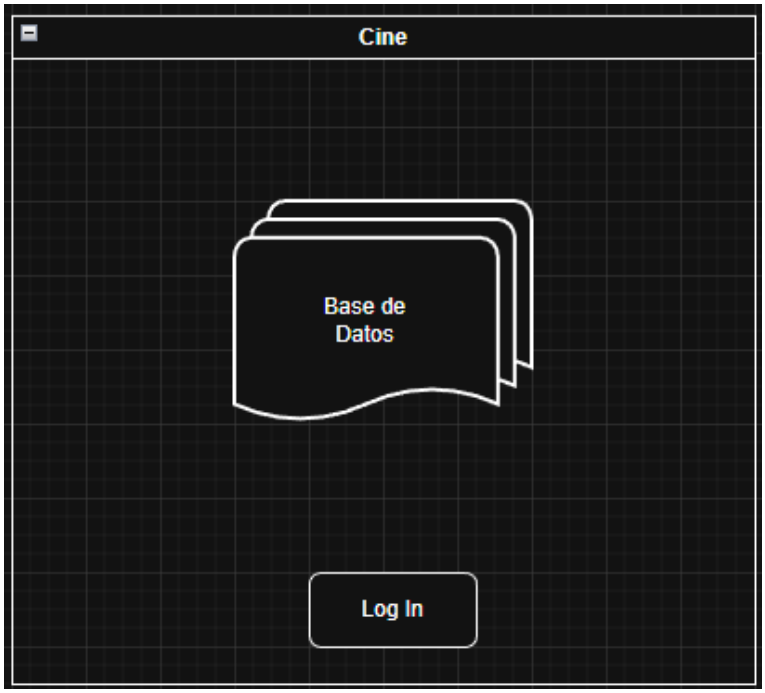
USUARIO BASE DE DATOS

Por motivos de seguridad hemos creado un usuario con los permisos justos y necesarios, y con acceso único a esta base de datos.

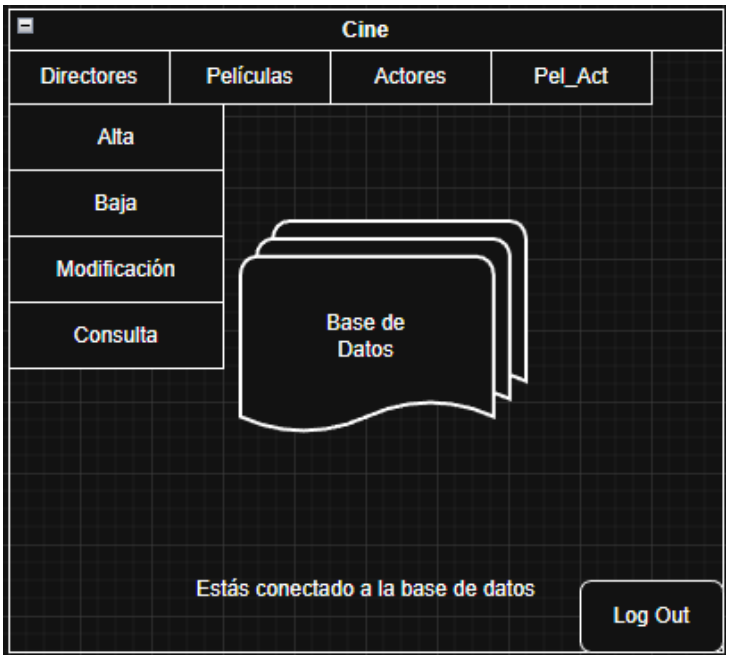
```sql
-- Crear usuario Select, Insert, Delete y Update (CRUD) localhost
CREATE USER 'studium'@'localhost' IDENTIFIED BY 'Studium2025#';
GRANT SELECT, INSERT, DELETE, UPDATE ON p_cine.* TO 'studium'@'localhost';
FLUSH PRIVILEGES;
```

DISEÑO FUNCIONAL

Principal



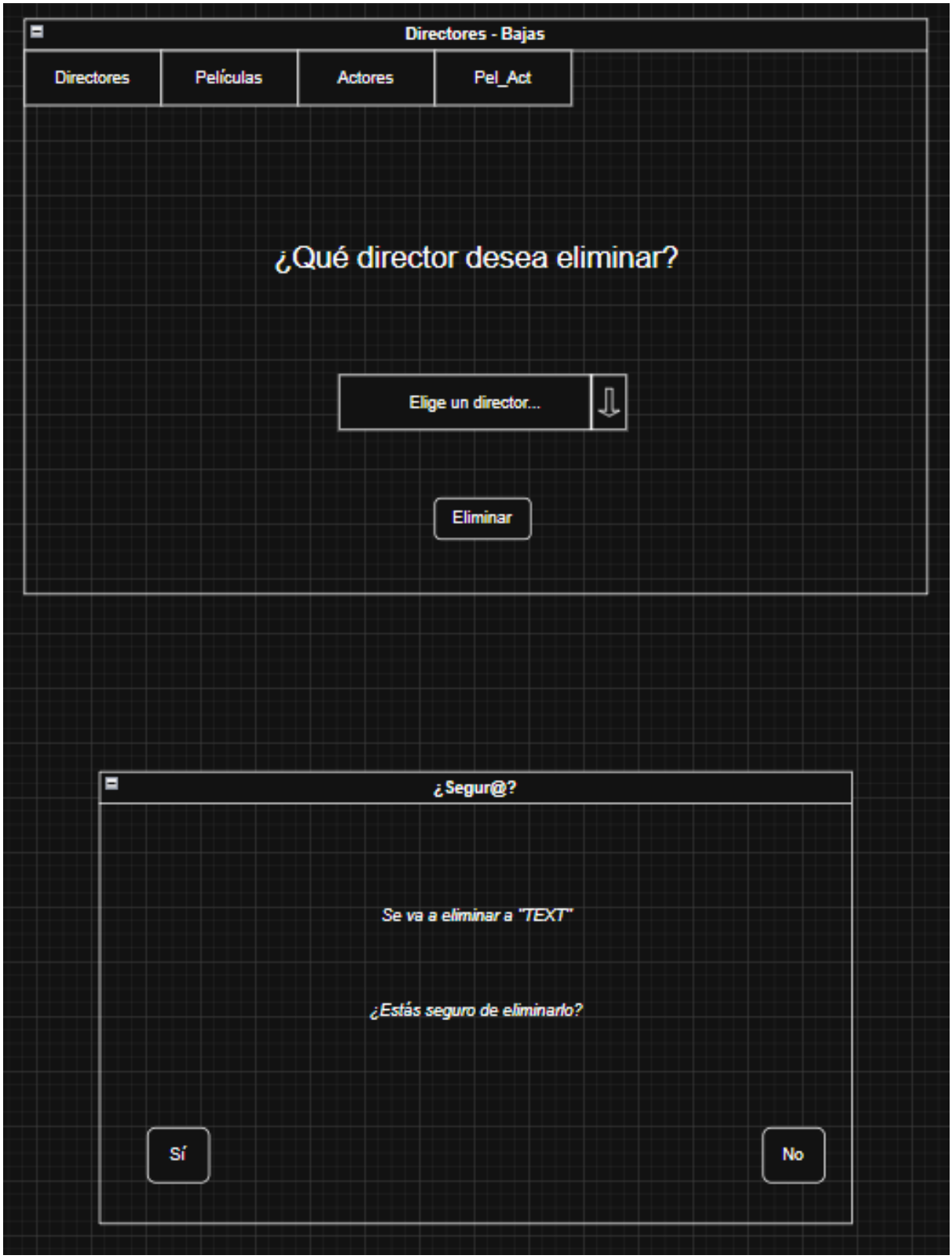
Principal2



LogIn

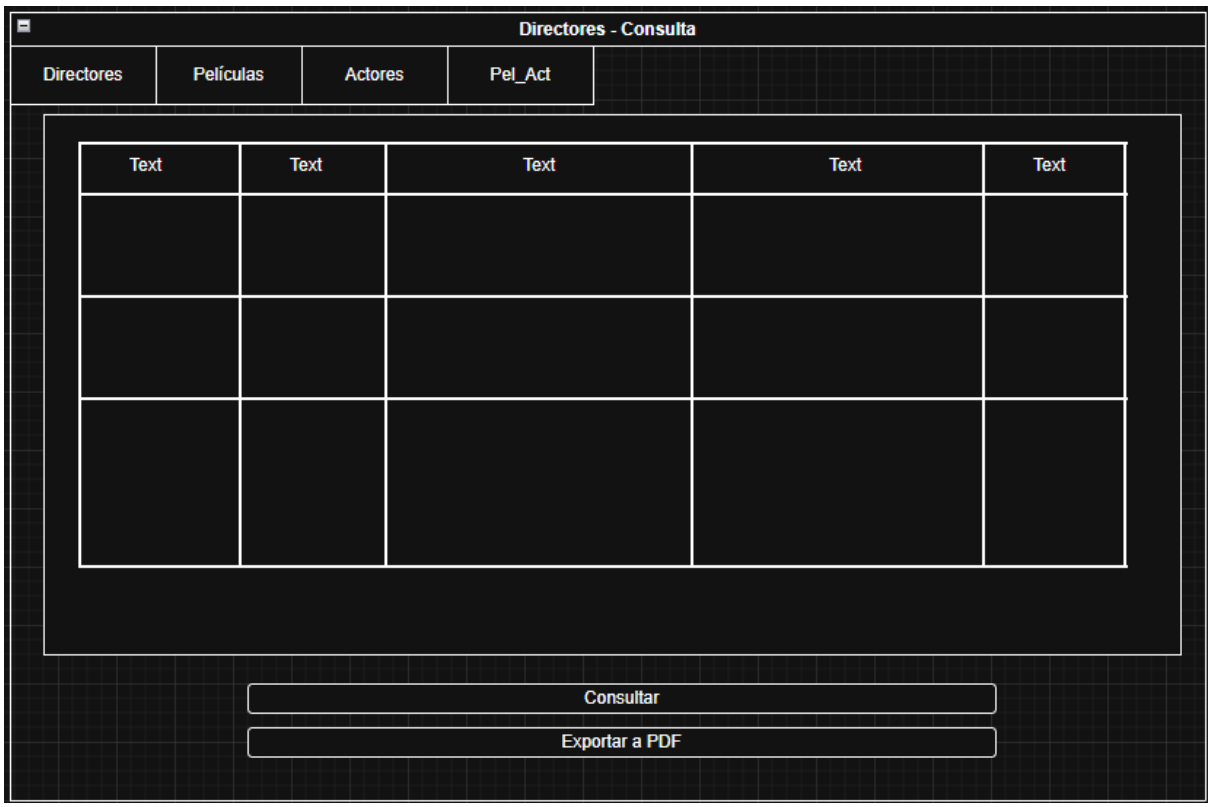
Directores – Alta

Directores – Baja



Directores – Modificación

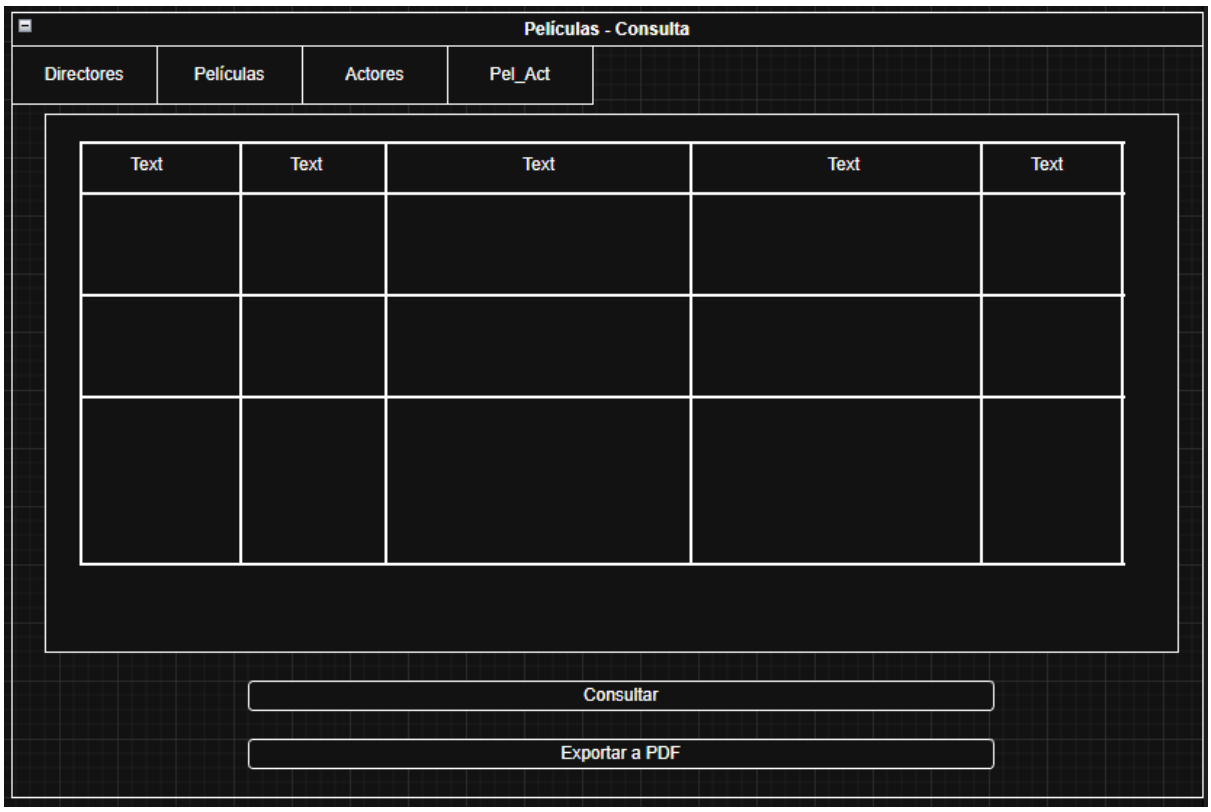
Directores – Consulta



Películas – Alta

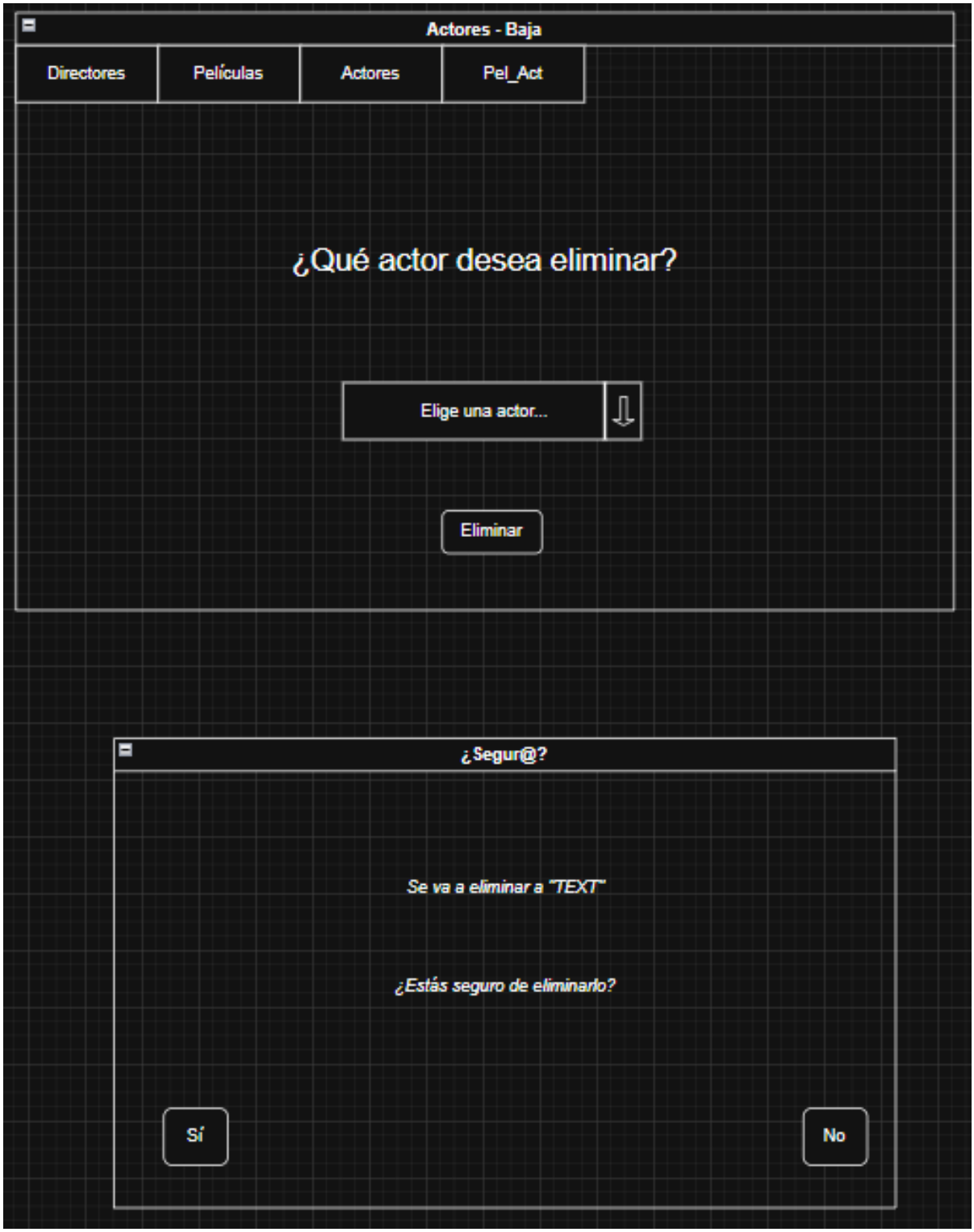
Películas – Baja

Películas – Consulta



Actores – Alta

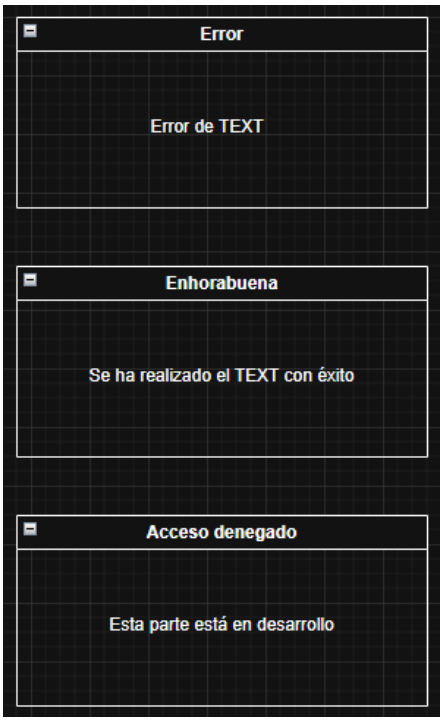
Actores – Baja



Actores – Modificación

Actores – Consulta

Diálogos



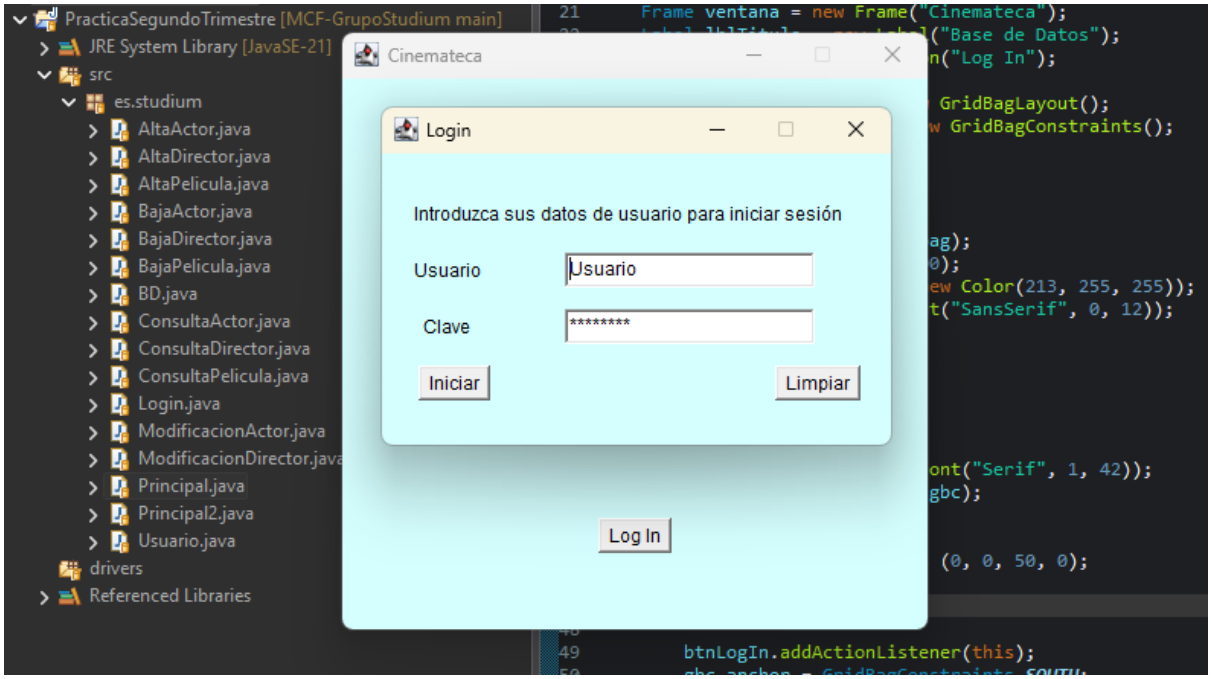
REPOSITORIO GITHUB

En este enlace se encuentra el repositorio dónde he llevado a cabo el proyecto, con todos los archivos que he ido usando para el desarrollo de la aplicación.

EJECUCIÓN Y PRUEBAS

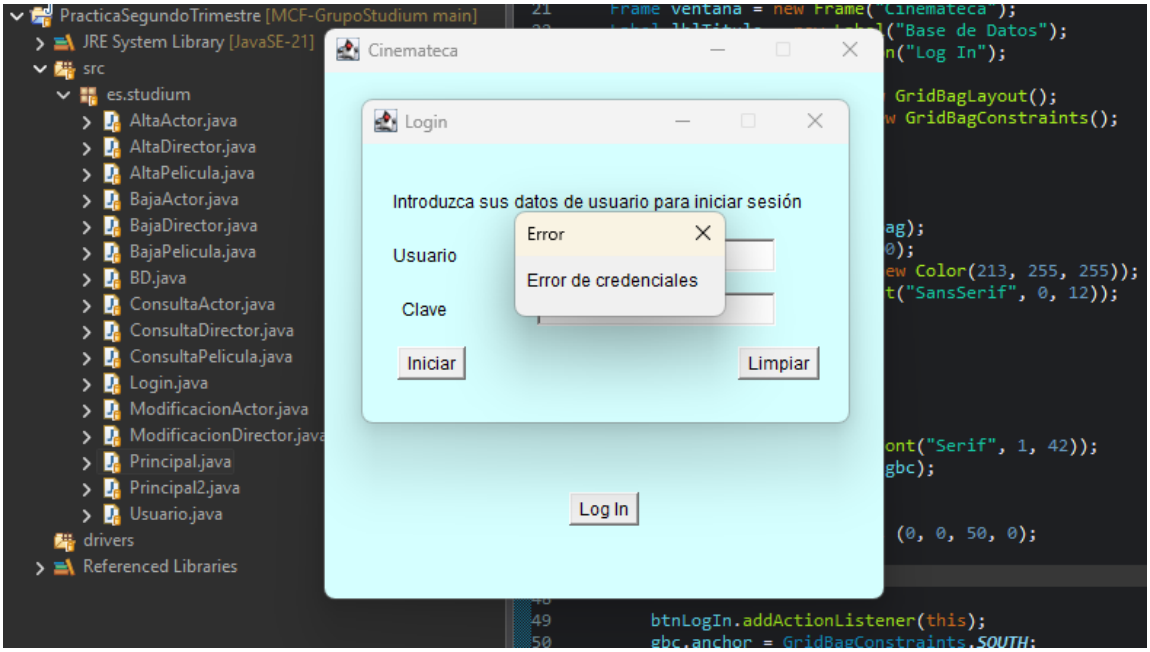
Cuando ejecutemos nuestra aplicación comenzaremos desde la clase JAVA Principal, que nos mostrará un Frame con un botón de LogIn, desde ahí al presionarlo nos enviará a la clase Login, que ejecuta otro Frame en el que debemos escribir los datos de inicio de sesión, esto realizará un SELECT a nuestra Base de Datos pasando varias comprobaciones para ver si coincide con alguno.

Actualmente disponemos de dos usuarios: “Usuario” que es de tipo “basico” y solo dispone de permisos para realizar altas (INSERT INTO) y de “Administrador” que es de tipo admin y dispone de todos los permisos del CRUD (sentencias SQL INSERT INTO, SELECT, UPDATE, DELETE). La clave está encriptada por motivos de seguridad.



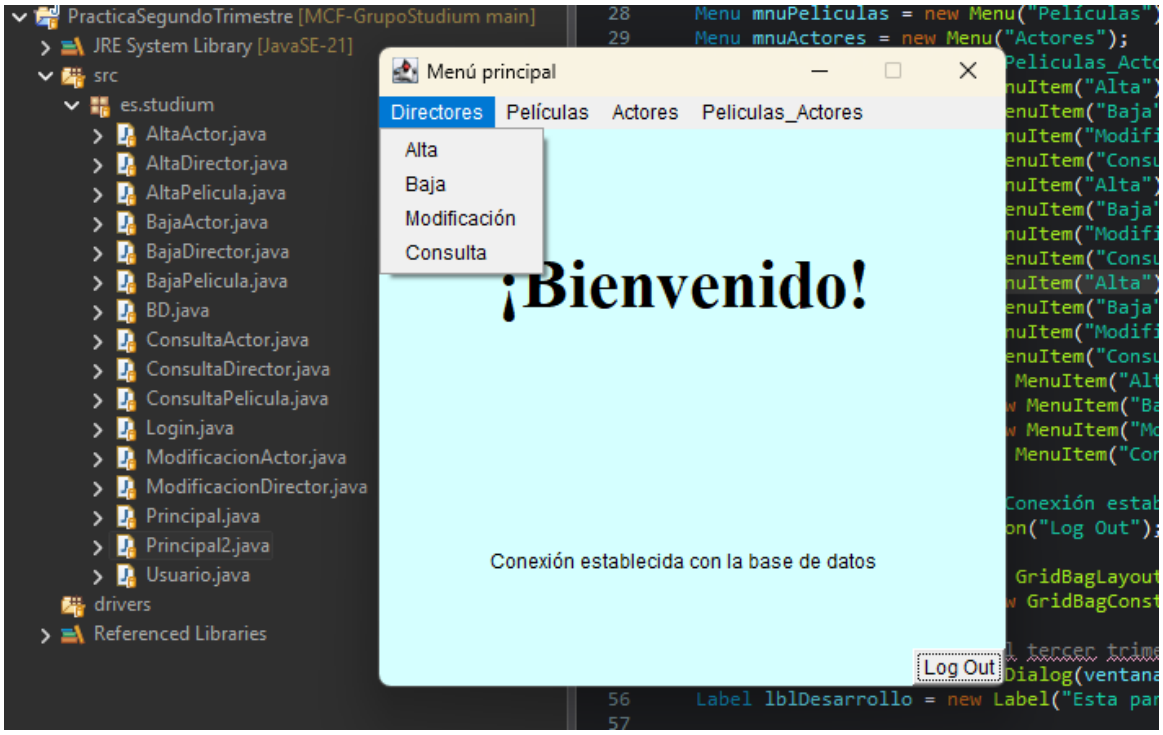
idUsuario	nombreUsuario	claveUsuario	tipoUsuario
1	Administrador	ab20f6e3ee225e70bdeb0363708de19b44a7e1...	admin
2	Usuario	ab20f6e3ee225e70bdeb0363708de19b44a7e1...	basico
NULL	NULL	NULL	NULL

Si escribimos los datos incorrectos nos saltará un Dialog indicando que ese usuario no existe en nuestra base de datos.

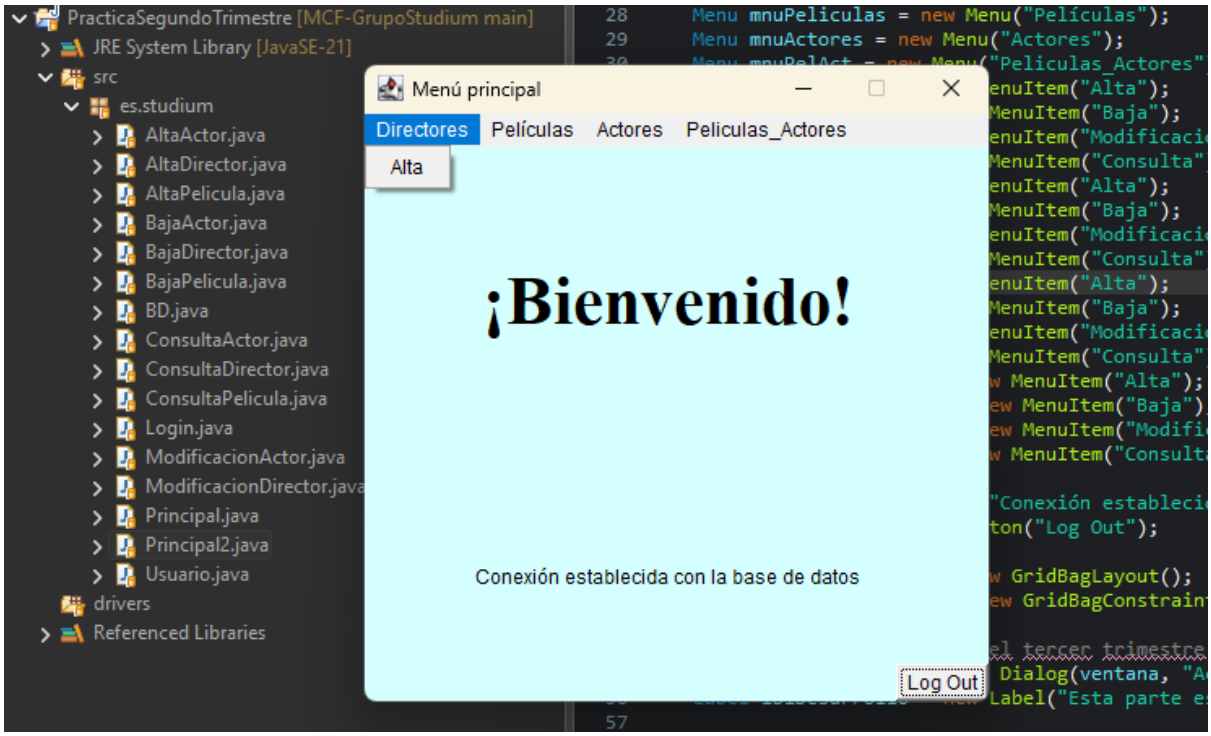


De otro modo si escribimos las credenciales de uno de los usuarios de nuestra Base de Datos accederemos a la parte funcional de nuestro programa.

Acceso Administrador

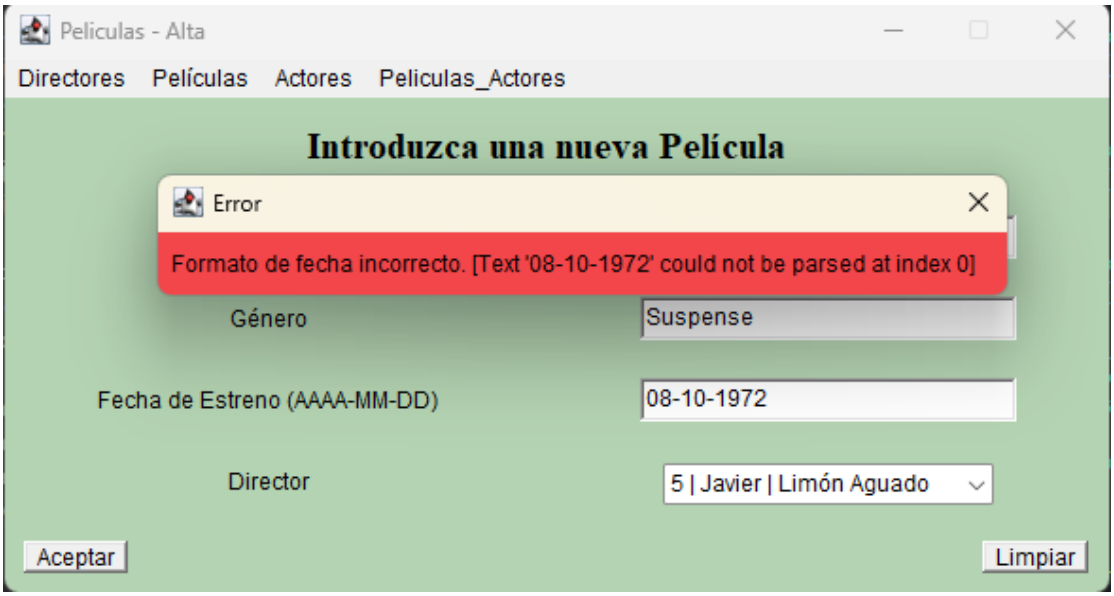


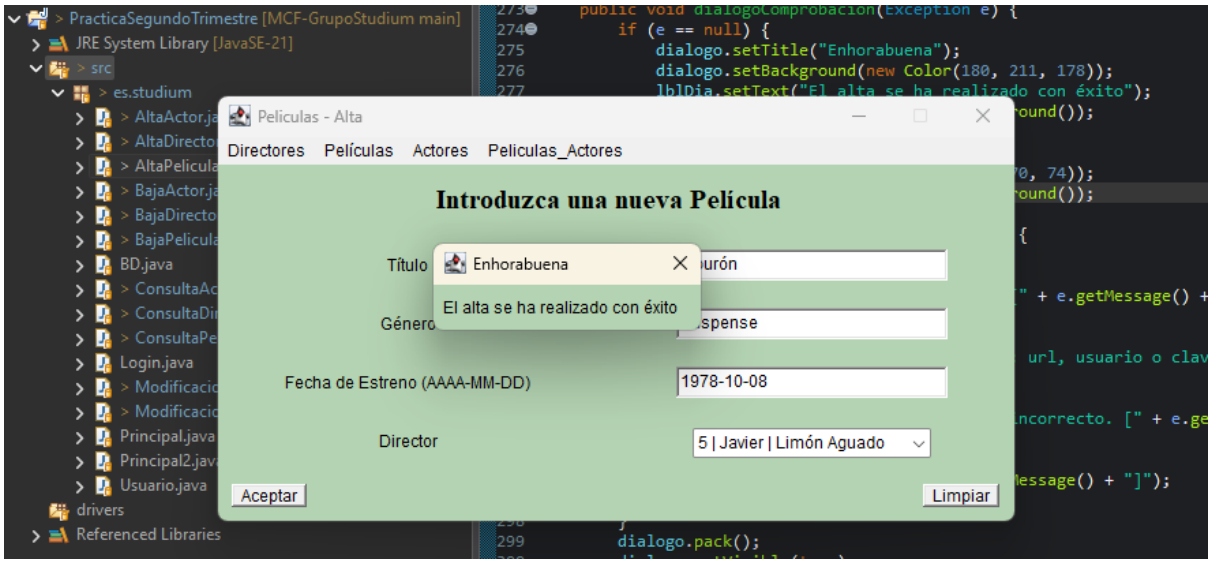
Acceso Usuario



En la barra de menú superior tenemos varias opciones, cuando presionamos una de ellas nos ejecutará la clase JAVA correspondiente, lo que abrirá un nuevo Frame.

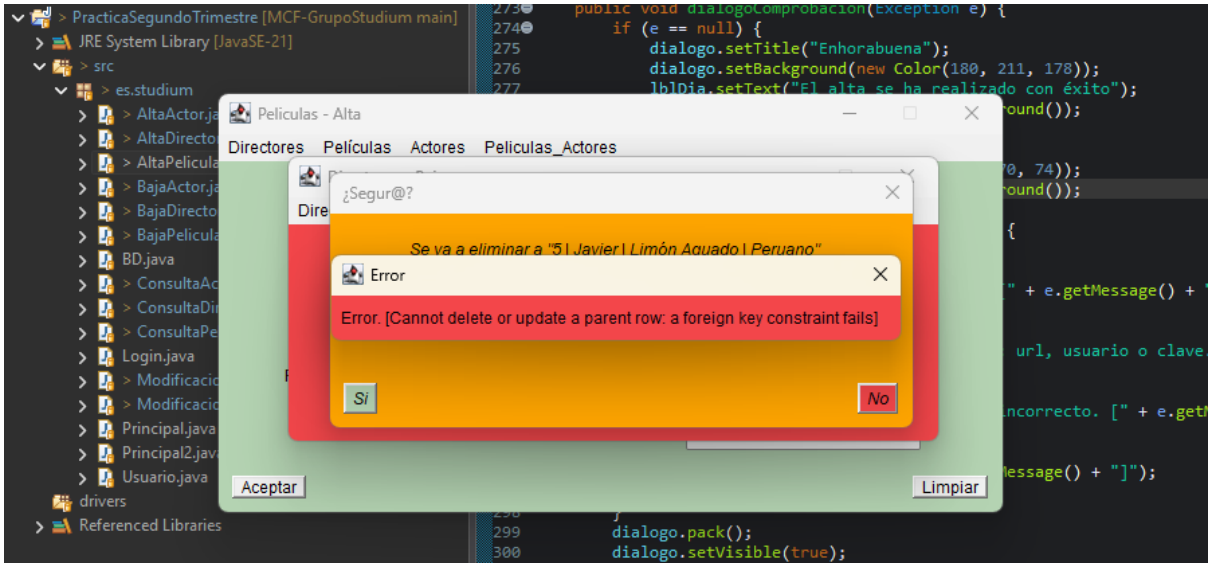
Tenemos varias verificaciones en nuestro programa, si realizamos cualquiera de las operaciones del CRUD de forma incorrecta, nos saltará un Dialog con el mensaje de error, por otro lado, si todo sale bien tendremos una confirmación de que la operación se ha llevado a cabo con éxito.



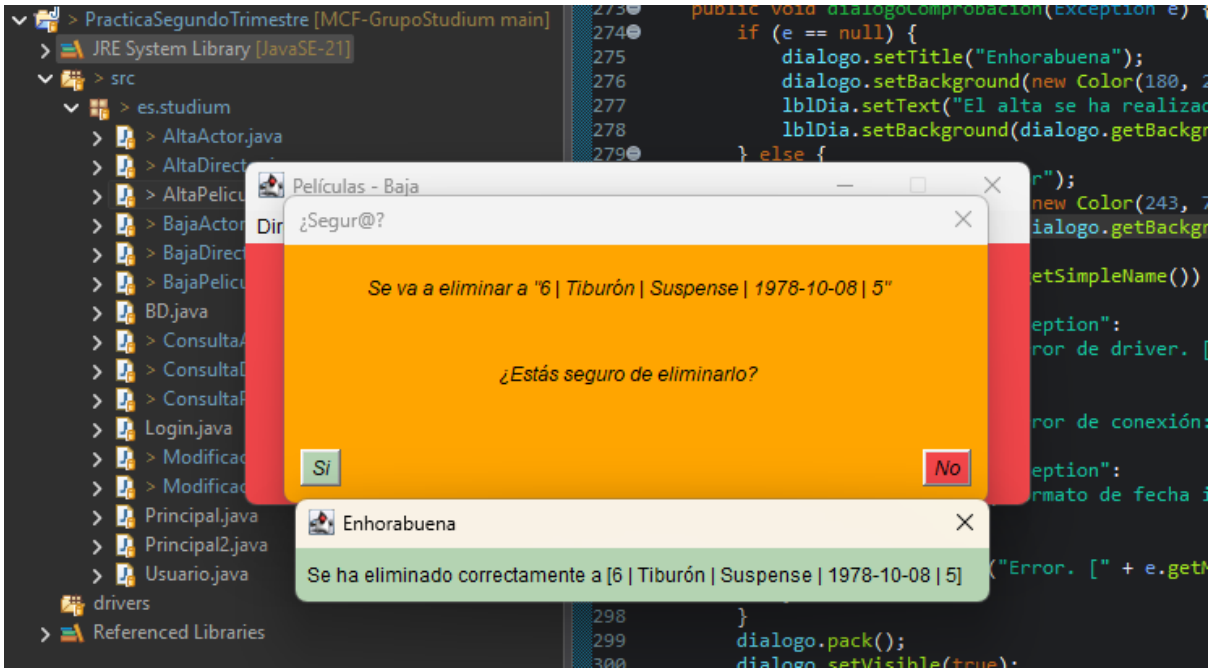


En el caso de las bajas cabe tenemos un Dialog de confirmación, y cabe destacar que para la tabla de Directores no se puede eliminar un Director esté asignado a una película, si lo intentamos saltará un aviso de error, ya que por nuestra configuración de la Base de Datos, ya que es una FK perteneciente a otra tabla, si queremos cambiarla deberíamos cambiar la configuración de "OnDelete" y "OnUpdate" a "Cascade".

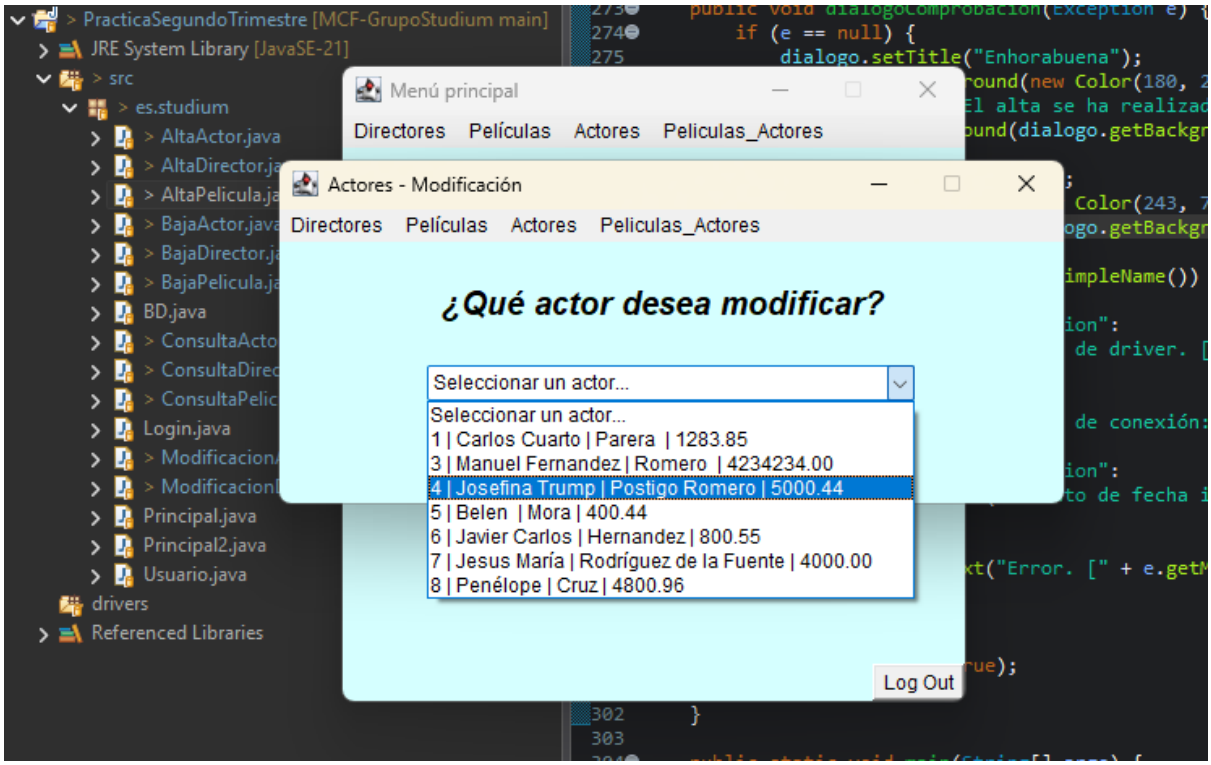
Para cada error hay un método creado que captura las excepciones y nos muestra el error en concreto.



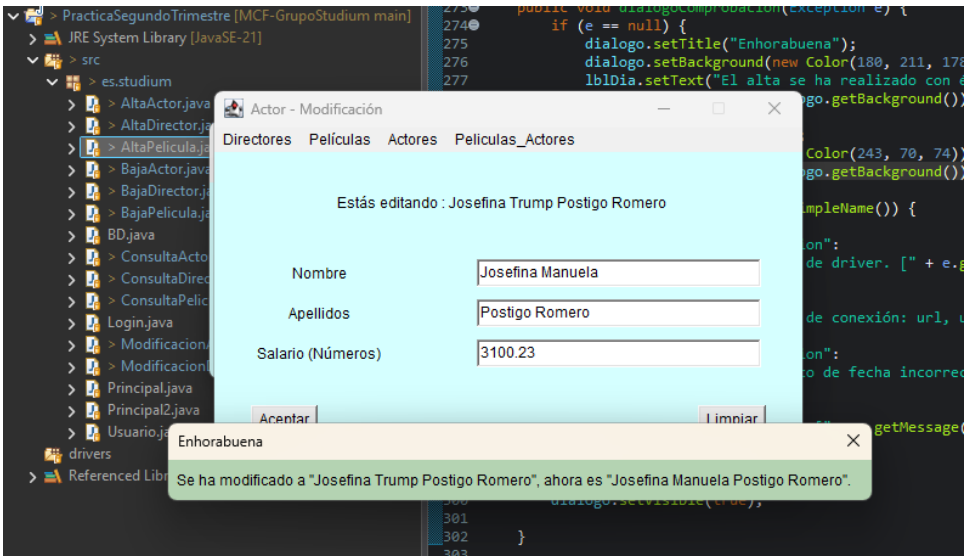
En cambio, si realizamos una baja con éxito tendremos nuestro mensaje de confirmación.



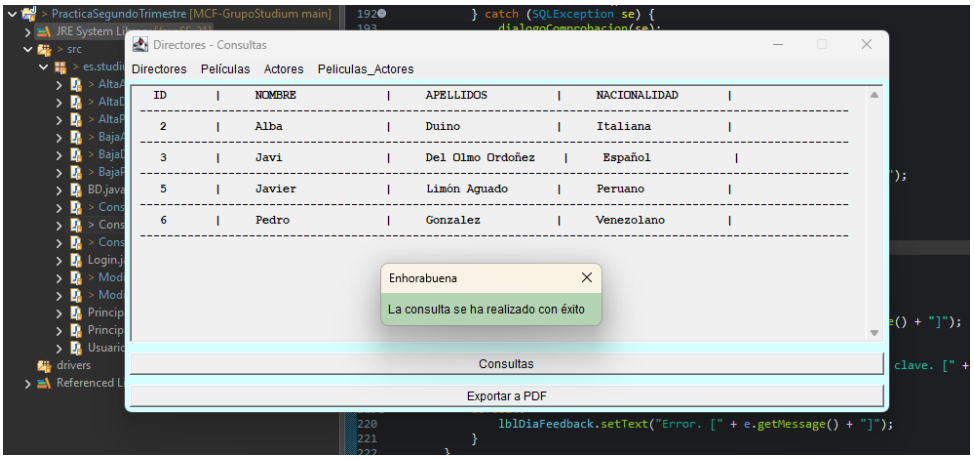
Para las modificaciones, igual que para las bajas disponemos de un Choice que nos permite elegir el elemento que queramos modificar.



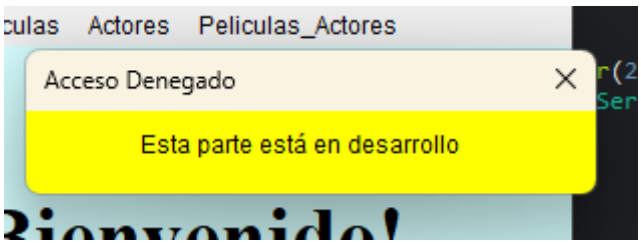
La modificación se actualizará a tiempo real cuando realicemos el UPDATE, mostramos la ejecución con éxito.



Mostramos también la ejecución de una consulta, en la que hemos creado una especie de tabla en un TextArea para que sea más agradable visualmente.



Tanto en el apartado de Películas_Actores como en Películas – Modificación, tendremos un Dialog con un aviso, ya que esta parte la desarrollaremos más adelante.



REFERENCIAS

- Apuntes de clase

- GitHub

- Documentación Oracle